# Claude Code Task: NODE Spatial Layer + Unified World Kernel + The Observatory

Read `docs/HANDOVER.md`, `CLAUDE.md`, `docs/BLUEPRINT.md`, and `docs/NODE_VISUAL_DESIGN_BRIEF_2026-08-07.md` before starting. Standard session rules
apply: work directly on `main`, log in `DEVLOG.md` as you go, keep `BLUEPRINT.md`
matching reality, rewrite

`HANDOVER.md` at the end, keep `README.md` current, push doc updates one at a time. All six standing design constraints in `CLAUDE.md` are binding on everything below.

**The goal of this task:** turn NODE from three disconnected mathematical models into one running, observable, deterministic world that a human can watch from above and walk through from inside. This is an *instrument*, not the shipping game client. Its purpose is to let design decisions be made against observed behaviour instead of intuition.

Build in the order given. Each phase is testable on its own and later phases depend on
earlier ones.

---

## The core diagnosis: NODE has no spatial primitive

Everything currently in `src/engine/` is aspatial.
`districtArrivalChoice()` resolves core-versus-periphery as a weighted coin flip and nothing persists afterwards. There is
no coordinate, no building, no plaza, no adjacency.

But multiple already-built or already-designed mechanics assume space exists:

- `src/comms/decay.ts` degrades rumours **by distance** — currently an abstract hop count.
- `detectionProbability()` and `patternSabotageAttempt()` roll **per witness** — witness counts are currently a passed-in number with no derivation.
- Proximity conversation (design addendum) is defined by **who is close enough to hear**.
- Fog-of-recognition is defined by **watching two people trade across the plaza**.

- District Weather and the Wall's Emissive Soul need **persistent per-district state**,
  explicitly flagged as unbuilt at the top of `ecosystem.ts`.
- The visual design brief maps game state onto **tiles, density gradients, and buildings**.

Build the spatial layer first. It is not a rendering convenience — it is the engine
primitive several existing mechanics are currently standing on top of as placeholders.

---

## Phase A — `src/engine/space.ts` (spatial

## primitive)

A pure, deterministic, dependency-free module in the same style as the existing engine
files. No rendering concerns, no client awareness.

**Model:**

- **Shard** — one node, target population 50-80. Contains districts.
- **District** — a *persistent entity with its own accumulating state*, not a coin-flip
  label. Minimum: a stable id, a classification (core or periphery), a set of plots, a
  population, and its own economic-health / detection /

weather history. This directly closes the "persistent per-district state is unstarted" gap flagged in `ecosystem.ts`.

- **Plot** — an addressable position within a district. Integer grid coordinates.
  A plot may hold a building, be a street, or be plaza ground.
- **Building** — bound to a role slot where one exists (a Miller's mill, a Baker's shop),
  so `vacancy.ts`'s slot states have a physical location. A building's visible state is
  derived from its slot state, not stored separately.
- **Plaza** — exactly one per district. The civic space. This is where

monuments live
(see Phase F) and where public witnessing concentrates.

**Required functions (pure, deterministic, seeded where random):**

- `distance(a: Plot, b: Plot): number` — walking distance, not euclidean-through-walls.
- `plotsWithin(centre: Plot, radius: number): Plot[]`
- `occupantsWithin(world, centre, radius): PlayerId[]` — **this is the function that finally derives witness counts** instead of accepting them as a parameter.
- `districtOf(plot): DistrictId`

- `generateShardLayout(seed, config): Shard` — deterministic procedural layout. Core district denser (tighter plot spacing, more buildings per unit area), periphery sparser, per the visual brief's density-gradient requirement. Same seed must always produce the identical layout.

**Wire the existing mechanics onto it, do not duplicate them:**

1. `detectionProbability()` and `patternSabotageAttempt()` should take a witness count **derived from** `occupantsWithin()` at the sabotage target's location. Report what this does to the existing

calibrations — a real spatial witness count will almost certainly differ from the assumed ~23, and that materially changes both the near-non-viable act-based numbers and the 146-220 day pattern-based proposal. **Report the new numbers; do not silently re-tune.**

2. `src/comms/decay.ts`'s hop distance should be derivable from real spatial and social distance rather than an abstract integer. Keep the existing decay curve; change only where the distance value comes from.

3. `districtArrivalChoice()` should place an arrival at an actual plot in an actual district, and that district should persist and accumulate state thereafter.

**Tests:** layout determinism under a fixed seed, distance symmetry and triangle
inequality, occupancy queries against hand-computed ground truth, and a regression test
proving the density gradient actually exists (core plot density measurably exceeds
periphery). Do not weaken any existing test to accommodate this.

---

## Phase B — `src/world/world.ts` (unified kernel)

Right now Phase 1 market, Phase 2 vacancy/conscription, and the ecosystem layer are
three separate islands that have never run together as one world. `HANDOVER.md` lists
wiring them as the next task. Do it here.

Build a single authoritative world object and one tick function:

```
createWorld(seed: number, config: WorldConfig): World
stepWorld(world: World):
```

```
World   // one tick, pure,
deterministic
```

The tick must step, in a documented and stable order: space/occupancy → vacancy and conscription → market (Miller then Baker) → ecosystem (health, experience, migration, sabotage) → comms (rumour propagation and decay). **Write the tick order down in `BLUEPRINT.md` and add a test that pins it**, because ordering changes will silently change every downstream number. `design/tick_order_check.py` already exists as prior art for this concern — check it before choosing an order.

Requirements:

- **Fully deterministic.** Same seed plus same config equals byte-identical state sequence. Use the existing `src/sim/rng.ts`; no `Math.random()` anywhere in the kernel.
- **A BACKSTOPPED or conscripted Miller must actually participate in pricing.** That is the specific unwired gap named in the handover.
- Existing engine modules are called, not reimplemented. If wiring reveals a genuine conflict between two modules' assumptions, **say so in `DEVLOG.md` and stop for review**

**rather than papering over it** —
that is the standing instruction.

**Expect this phase to surface contradictions.** Three models calibrated in isolation
rarely compose cleanly. Finding and reporting those contradictions is a primary
deliverable of this task, not a failure of it.

---

## Phase C — Synthetic drivers (test harness only — read this carefully)

To run a world you need occupants making decisions. This is in direct tension with

standing constraint 3 ("does this need to be an agent") and with the project's
foundational rule that no AI actor exists in NODE.

**The resolution, which must be documented and enforced:**

Synthetic drivers are **test instrumentation, never game content**. They exist to
exercise the engine the way a load generator exercises a server. They are deterministic
policy functions — a small set of fixed strategies (`honest`, `opportunist`,
`saboteur`, `idle`) chosen by seed, each a pure function from visible state to action.

No learning, no belief modelling, no personality, nothing that infers intent.

Enforce this structurally, not just by convention:

- They live in `src/sim/drivers/` only.
- **Add a test that fails if anything under `src/engine/`, `src/world/`, or `src/server/` imports from `src/sim/drivers/`.** This is the guardrail that stops test scaffolding from quietly becoming shipped NPCs.
- Document in `BLUEPRINT.md` that these are harness-only and why.

Each driver's action space must be limited to what a real player can actually do:
occupy or vacate a role slot, set a price, move between plots, speak in the constrained
grammar, propagate a rumour, and attempt sabotage steps.

---

## Phase D — The snapshot contract (`src/world/snapshot.ts`)

One versioned, serialisable structure describing the complete visible state of the
world at one tick. **This is the single contract every view consumes.** Get

it right;
everything downstream depends on it.

Must include: schema version, seed, config hash, tick number and simulated date, the
shard layout (emitted once in a header, not per tick), per-district state (population,
economic health, weather, monuments), per-plot and per-building state, per-player
position and role and visibility tier, market state (prices, volumes, flour cost),
vacancy slot states, active rumours with their current fidelity, and any in-progress
sabotage pattern legibility.

Two transports, same schema:

- **Replay file** — newline-delimited JSON to disk. `npm run world-record -- --seed 42 --days 90` produces a complete replayable run.
- **Live stream** — extend `src/server/ws.ts` to broadcast snapshots. Preserve the existing per-player targeted `RumourMessage` behaviour; the observatory connects as a privileged observer that sees everything, and **that privileged view must be impossible for a normal player connection to request.** Add a test for that.

Add a determinism test: record a run, replay from the same seed, assert byte-identical
output.

---

## Phase E — The Observatory (`observatory/`)

A local web application. **This is deliberately not the Godot client.** Godot remains
the eventual real client; this is a diagnostic instrument that needs to be fast to
build, fast to change, and verifiable headlessly. Keep it in its own directory with its
own `package.json` so it never

entangles the engine's dependencies.

Stack: Vite + React + TypeScript + Three.js (`@react-three/fiber` and `@react-three/drei` are appropriate). It must consume the Phase D snapshot schema and nothing else — no direct engine imports, so the contract stays honest.

**One scene, two cameras:**

1. **Top-down** — orthographic, near-isometric, the whole shard visible. Buildings as
   massed geometry, districts legible, plaza distinct, players as moving markers.
   Overlay toggles for: economic

health heatmap, vacancy states, rumour propagation, witness density, sabotage pattern legibility.

2. **First person** — same scene, walk the streets at ground level. WASD and mouse-look is fine. The purpose is to answer questions the top-down view cannot: does a plaza at this density feel crowded or empty, can you actually see two people trading across it, does the core read as different from the periphery when you're standing in it.

Toggle between them with a single key. Same underlying scene graph —

do not build two
renderers.

**Controls:** timeline scrubber across
the whole recorded run, play/pause,
speed
control, seed entry with re-run, and a
live-connect option pointing at the
WebSocket
server.

**Visual rules — these are binding, not suggestions.** Follow `docs/NODE_VISUAL_DESIGN_BRIEF_2026-08-07.md` and `docs/DESIGN_ADDENDUM_2026-08-08.md` exactly: colourless near-white daylight (not warm
amber), saturated colour reserved
strictly for genuine heat sources,
and the Visual

Contrast Contract honoured. Getting the light wrong here will produce misleading
conclusions about how the world actually reads. Placeholder geometry is entirely
acceptable; placeholder lighting is not.

**Verification:** run it headlessly against a real recorded snapshot file and confirm it
loads, renders a frame, and steps the timeline without errors — the same discipline
applied to the Godot client on 2026-08-07. Note explicitly in the handover what a
headless check cannot confirm.

---

## Phase F — Civic memory: the plaza and monuments

Standing constraint 4 now reads: personal memory is mortal, civic memory is immortal.
Civic memory currently has nowhere to live. Give it one.

Build a minimal, honest version:

- A **monument** is a persistent record of a public, collectively-witnessed *event*,
  anchored to a plot in a district's plaza. It never expires.
- The eligibility test is exactly constraint 4's: it must record an event the node

collectively witnessed, never an individual's expression or private judgement. A
detected sabotage campaign, a shard-threatening vacancy, a conscription — candidates. Anything one player said or felt — never.

- Monuments must be visible in both observatory views, and legible as accumulating
history in the top-down view over a long run.

**Do not build a reputation system in this task.** Constraint 6 governs it and no
design exists yet. If you find yourself needing a reputation value to make

something
work, stop and flag it.

---

## What NOT to build in this task

Stay out of these; each needs its own design pass and several are explicitly blocked:

- Reputation scoring of any kind.
- The Oracle, the private diary, proximity conversation, postcards/exit tickets — all still `[DESIGN — not yet built]`. You may surface their *state* in the snapshot schema
  as reserved fields, but do not implement their mechanics.

- Phase 5 voice/safety. Hard-gated pending legal review.
- Any change to the Godot client beyond leaving it working.
- Adopting the pattern-based sabotage proposal as default — Steven's call, still open.
- Adjusting `R`/`N` vacancy defaults — blocked on the role roster.
- Resolving `TRAVEL_DAYS_TARGET` — explicitly Steven's decision.

---

## Deliverables

1. `src/engine/space.ts` with tests, and the three existing mechanics wired onto real spatial data, **with a written report of what the real witness counts**

**do to the**
**sabotage calibrations**.

2. `src/world/world.ts` — unified deterministic kernel with a pinned, documented tick
   order, and a Miller that actually prices when backstopped or conscripted.
3. `src/sim/drivers/` — harness-only synthetic drivers, with the import-guard test.
4. `src/world/snapshot.ts` — versioned schema, file recording, live streaming, observer
   privilege test, determinism test.
5. `observatory/` — dual-camera local web app, headlessly verified, visual brief honoured.

6. Monuments and plaza as the civic-memory channel.
7. All existing 83 tests still passing; `npm run typecheck` clean.
8. New scripts wired into `package.json`: `world-record`, `world-replay`, `observatory`.
9. Docs updated per the standing rules, including a plain "how to run the observatory" section in `README.md`.

## Report back explicitly on

- Every contradiction surfaced by composing the three previously-separate models.
- What real spatial witness counts do to both sabotage calibrations.

- Whether the density gradient produces a core that genuinely reads differently from the periphery when standing in it, or whether the current numbers make them indistinguishable.
- Anything the spatial layer reveals about mechanics that were only ever validated aspatially.

Flag concretely and keep moving where something is not blocking. Where something *is* blocking, stop and say so rather than working around it silently.